Hadoop

1.Hadoop是什么，优缺点。

Hadoop是一个分布式系统架构，由HDFS，MapReduce，yarn组成。

优点：

1. 高可靠性：多副本机制。
2. 高扩展性：通过拓展廉价的机器提高整体集群的性能。
3. 高效性：MR框架将一个大型的任务分成多个小任务进行执行。
4. 高容错性：任务失败之后自动在其他节点重启任务。

缺点：

1. 不适用低延迟访问。
2. 不能高效地处理小文件。
3. 不支持多用户写。

2. HDFS的定义。

HDFS是一个高可靠，高拓展的分布式文件系统。采用主从和主备的设计模式。由以下三部分组成：

NameNode：存储元数据信息。

SecondaryNamenode：用于帮助Namenode分担压力，合并Fsimage和Editlog。

DataNode：存储实际的数据。

3. Hadoop的读写文件流程：

读文件：

1. 客户端向Namenode发起读文件的申请，用户获取数据所在的位置。
2. 客户端根据数据所在的位置就近向DataNode读取数据。
3. DataNode以Packet为单位向客户端传输数据。

写文件：

1. 客户端向NameNode发起写文件的申请，Namenode返回父目录是否存在等相关信息。
2. 如果存在则返回写入文件的DataNode地址。
3. 客户端在各个DataNode之间建立一条通信管道，以Packet为单位来传输文件。

4. MR的任务执行流程。

Split阶段：切分文件，其中切分之后的每一个文件对应一个Task。

Map阶段：将文件的数据切分成KV键值对的形式。

Shuffle阶段：

- Map端shuffle：KV键值对被放入环形缓冲区，当达到80%的内存空间的时候对里面的数据进行快排并且溢出到磁盘中生成小文件。当生成足够多的小文件的时候进行归并排序生成一个有序的大文件。
- Reduce端shuffle：Ruduce根据自己的分区向DataNode去获取自己对应的文件，然后进行归并排序将生成一个有序的大文件。

Reduce阶段：各个Reduce根据得到的有序大文件执行运算得到最终结果。

5. yarn是什么，三种资源调度器的区别是什么?

yarn是一个资源管理器，由四部分组成：ResourceManager，ApplicationMaster，NodeManager，Container。执行任务流程如下：

1. 用户将应用程序发送到RM
2. RM为AM申请资源，向NodeManager提出启动Container来启动AM。
3. AM和RM进行通信来申请资源，之后再与NM通信启动Task。
4. 所有任务结束之后，AM向RM进行注销。

三种常见的调度器：

先进先出调度器：根据任务来的顺序分配资源，会导致紧急的任务无法得到及时的处理。

容量调度器：设置每个队列的最小容量，来保证小任务能够得到充分的资源。比如为每一个部门来设置容量大小。

公平调度器：提前为每一个队列来设定权重，保证重要的任务能够得到足够的资源。

Hive

1.hive是什么，主要架构是什么?

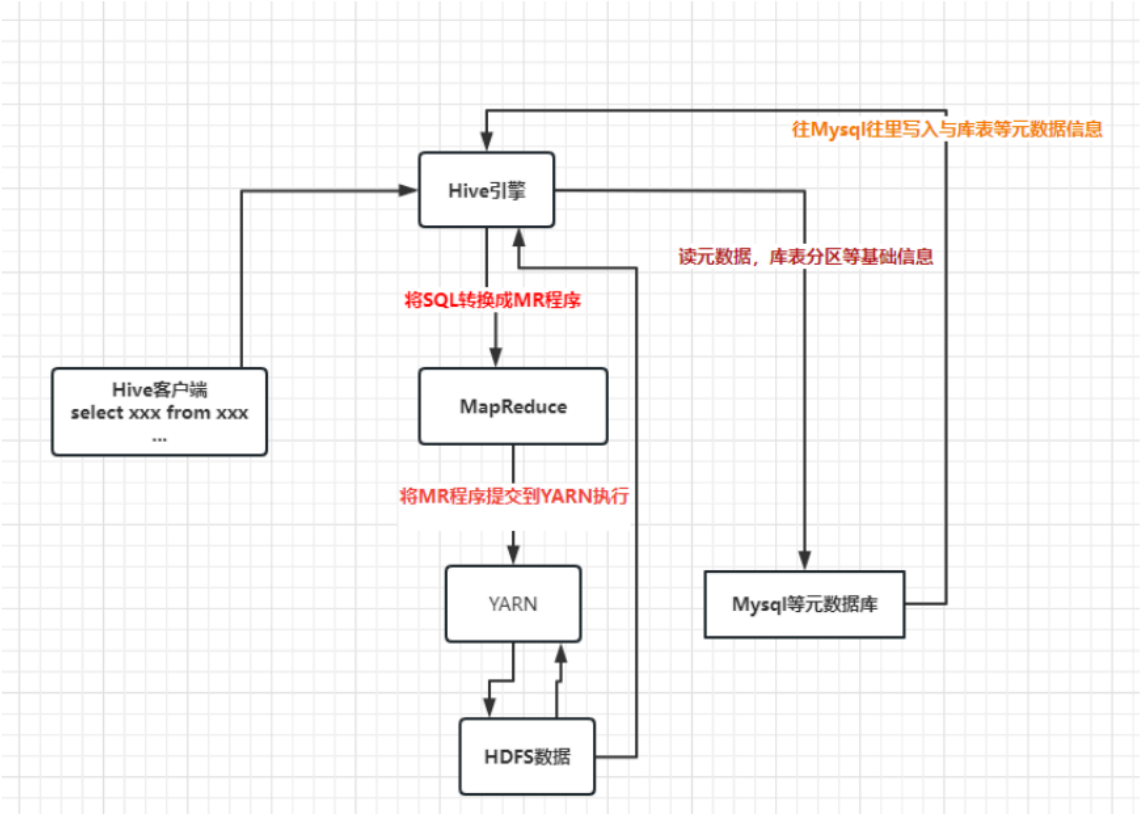
hive的本质就是将HQL转换成MR任务。由客户端和服务端两部分组成：

客户端：CLI，GUI。

服务端：Driver（Compiler，Optimiter，Executor），metastore，Thrift Server（其他语言的接口）

2. hive的任务执行流程

Hive和Hadoop的交互关系如下所示：得到客户端之后sql之后Hive引擎结合元数据信息来生成MR任务，交到Yarn进行执行，最后得到结果。



Hive的任务转换执行流程：

先调用Driver生成语法树。

检查语法树是否存在问题。

根据语法树来生成逻辑执行计划。

通过元数据来优化逻辑执行计划。

将逻辑执行计划转化成物理执行计划，生成MR任务。

3. spark比MR快的原因是什么?

- Spark的落盘次数比MR少：spark只在发生宽依赖的时候才会执行shuffle操作，而每一个MR任务都会执行shuffle。
- Spark的Bypass操作：只在特殊场景下进行排序操作。
- Spark的Persist：可以对常用的数据进行持久化操作。
- Spark是线程级别的任务：线程级别的任务可以减少任务拉起所消耗的资源，可以共享进程中的资源。

Hive优化

1. SQL优化

过滤模式：

- 分区表：如果where中的字段属于分区字段，那么会在map前进行过滤，减少扫描量。
- 谓词下推：如果是外连接，主表中的字段应该放在where中，从表中的字段应该放在on中；如果是全连接，谓词下推失效，使用两个子查询进行连接；如果是内连接，默认在Map端进行过滤。
- 列裁剪：在select中过滤需要的字段，不会减少扫描量，减少之后任务的扫描量。

聚合模式：

conut(distinct 字段)：这种情况下之后产生一个reduce，效率低；使用group by+conut的方式将其划分为多个reduce进行处理。

order by：order by只会产生一个reduce任务。使用distribute by+sort by来产生多个reduce任务进行排序。

join模式：

map join：小表关联大表。

Bucket-mapjoin：同为分桶表，并且分桶数量为倍数关系。

SMB-mapjoin：在Bucket-mapjoin的基础上，sort by的字段也是分桶字段。

skew-join（只支持inner join）：自动根据热点键来进行负载均衡，划分到多个reduce中进行处理。不过和

2. 数据倾斜问题

数据倾斜问题分为map端和reduce端的数据倾斜。

map端的数据倾斜是因为输入的文件大小不均匀且文件不可拆分。解决方式是map端小文件合并和修改文件的压缩格式（ORC或者parquet）。

reduce端的数据倾斜是因为热点键问题，常见的解决方法就是负载均衡：

- inner join的情况下可以使用skew join。
- 其他情况下可以使用加盐处理热点值再union all。
- 小文件合并问题

关键参数：

- hive.merger.smallfiles.avgsize：输出的文件平均大小。
- mapred.min.split.size.per.rack：合并文件的最小大小。

MR任务之后会判断输出文件平均大小是不是小于avgsize，如果小于的话另起一个map-only任务，根据min.size来合并小文件。

存量小文件：直接使用insert overwrite来合并小文件。

4. 参数调优：

map和reduce的内存和核数调优：一般是1核2g。

map个数的控制：min.size和输入文件的个数以及文件是否可以拆分。

reduce个数的控制：可以直接指定reduce的个数；每个reduce处理的数据大小；reduce最大数量。

group by优化：map端聚合。

slow-start：在map端任务执行到指定进度时开始reduce的任务。可能会发生抢占资源，反复kill任务的情况。

Spark

1.hive和spark怎么控制生成文件的个数。

hive生成文件的个数等于reduce的个数，可以通过设置reduce的个数；每个reduce处理的数据量大小来控制。

spark输出的文件个数等于最后的分区数，控制小文件可以使用repartition来对分区进行限制。

2. Spark的AQE。

AQE的全称是自适应查询，根据前一个stage得到的数据信息来优化之后的stage执行计划。并不是所有情况下都会触发，只有当一个任务产生Shuffle的时候才会触发。

作用：

- 动态调整不同stage的partition个数。
- 动态优化数据倾斜：自动检测热点键将其拆分成多个分区。
- 动态join类型转换：广播表和SortMergeJoin之间进行转换。
- Spark的join类型有哪些?

Broadcast Hash Join：小表关联大表。

Shuffle Hash Join：两张表先根据join key进行分区，然后再将分区小的表作为Hash table进行连接。

Sort Merge Join：Spark默认的join方式，先根据join key进行分区，再对分区内的数据进行排序，再将相同分区的数据进行连接合并在一起。

笛卡尔积连接：当join key为空的时候触发。

广播嵌套循环连接：当小表关联大表时且非等值连接时。使用小表循环去每个Executor中执行join连接。

数仓理论

1.需求开发流程。

提出需求：业务方提出需求

需求分析：产品经理来进行需求分析

需求承接：简单需求承接和复杂需求承接（参考项目问题第7点）。

需求评审：如果是很简单的需求的话，就直接和业务那边面对面进行一个需求对接；如果需求设计的内容较多并且比较复杂的话，那就会拉伸产品经理、负责人和业务方进行一个业务需求会议。

ETL开发设计：

- 表模型的设计：设计表名、字段名；设计表的主键和外键；设计字段注释；设计表分区等。
- Mapping文档的设计：每个字段的生成逻辑；表与表之间的关系，数据链路设计。

  将上述的内容都产出ETL文件进行留存，为之后的开发流程做参考。
- 调度设计：根据具体的上下游依赖关系来设计调度任务。其中一个节点只生成一个表；多并行，少串行。

自测：

- 检查关联关系和过滤条件是否正确：join采用的是全连接还是左外连接；on的条件是否正确；where的条件是否正确；

- 检查对口径指标的计算是否正确：数据指标分为可累加和不可累加的数据口径；可累加的数据口径使用sum；不可累加的使用count。
- 检查是否能够进行优化：比如列裁剪，分区裁剪，谓词下推，避免使用count(distinct)等。

数据验收：业务方检查是否符合自己的一个需求。

上线。

2. 数据仓库的特点和好处。

特点：面向主题的，集成的，相对稳定的，能够反映历史趋势。

好处：

- 集成了多个数据源，提供了统一的视图。
- 保存历史数据，能够反应历史变化趋势，为业务决策提供了支撑。
- 采用了数据分层，提高了数据的可复用性。
- 数据仓库和数据库的区别。

数据仓库的出现不是为了替代数据库，而是数据库发展到一定程度后的必然产物。

- 设计目标：数据仓库是OLAP型，是为了分析数据；数据库是OLTP型，用于处理事务。
- 数据处理方式：数据仓库注重ETL，不会轻易的修改数据；数据库常见的操作是增删查改。
- 应用场景不同：数据仓库用于BI数据报表；数据库一般用于web应用开发等。
- 用户群体不一样：数据仓库的用户群体是数据分析人员；数据库的用户群体是开发人员。
- 两种数仓架构的区别。
- 设计思路：Immon采用自上而下的方式，以数据作为驱动；Kimball是自下而上的，以业务需求为驱动。
- 建模方式：Immon采用3NF建模，减少数据冗余，来提高数据的准确性和一致性；Kimball采用维度建模的方式，通过冗余数据来提高查询效率。
- 周期：Immon需要较长的周期，使用大公司；kimball以业务需求为导向，短平快，使用小公司。
- Kimball的灵活性更强。
- 数仓分层的作用。
- 屏蔽了原始数据的干扰。
- 提高的数据的准确性和一致性。
- 增加了数据的可复用性，降低存储成本。
- 保证数据血缘关系的清晰。
- 降低了开发和维护的成本。
- 减少烟囱式开发。
- 数仓层次调用的规范。
- 不能跨层调用。
- ads层不能相互调用。
- 任务的层数不能过深。
- 两种数仓建模的区别。

范式建模（减少数据冗余来提高数据的准确性和一致性，适用于OLTP）

维度建模（通过增加数据冗余来提高查询效率，适用于OLAP）

8. 维度建模的四个步骤。

确定业务过程

确定粒度大小

确定维度

确定事实

9. 总线矩阵的作用。

总线矩阵的横坐标是业务过程，纵坐标是维度。

- 为企业提供了统一的数据视图，避免重复开发。
- 指导分层建设：可以从总线矩阵中快速找到需要的维度。
- 支持跨主题域进行分析：不同的主题域可以通过相同维度进行分析。
- 为新人提供了一个快速了解公司业务过程的文档。
- 宽表的好处和局限性。

好处：

- 提高查询性能。
- 提高开发效率。
- 有效的解决口径不一致的现象。

坏处：

- 生产时性能较差。
- 开发难度高。
- 维护成本高。
- 稳定性较差。
- 数据模型设计的基本原则。
- 高内聚低耦合：将相同业务过程放在同一个主题域中。
- 核心模型和边缘模型分离：可以将核心字段放在一起，非核心字段另外进行存储。
- 公共处理逻辑下沉：将ADS需要用的公共指标放在公共层中。
- 口径一致性。
- 命名清晰。
- 如何保证指标的一致性。
- 命名一致性：使用规定的命名规范。
- 口径一致性：将指标的口径维护到平台上进行管理。

表命名相关规范：层级+数据域简称+业务过程+分区类型

13. 星型模型和雪花模型的区别。

星型模型是一张表与多张维度表直接关联。

雪花模型是维度表之间也会关联，再与事实表进行关联。

优缺点：

- 数据冗余。
- 查询性能。
- 设计复杂度。
- 维护成本。
- 数据一致性。

维度	星型模型	雪花模型	星座模型
数据冗余	高（非规范化维度）	低（规范化维度）	中等（共享维度可能冗余）
查询性能	高（宽表，少JOIN）	低（多表JOIN）	中等（共享维度减少部分JOIN）
结构复杂度	简单	中等	高
维护成本	低	中等	高
适用场景	单一业务过程快速分析	多层级维度分析	企业级多业务过程整合

14. 强规则和弱规则：强规则会直接终止任务；弱规则会直接进行相应的报警处理。
15. DQC的配置：表级别和字段级别。
16. 大概多少百分比的任务会触发DQC报警：4%以内都可以，一般SLA报警会比准确性的报警要多一点。
17. 数据域的划分遵循的原则：

- 高内聚低耦合：将相关的业务过程划分到一起
- MECE原则：数据域保持互斥；能够包含当前所有的业务过程；考虑之后可能加入进来的业务。
- 维度退化一般是在哪里?

一般维度退化会发生在dwd层和dws层。我们将常用的维度退化到事实表中：想要减少join的次数从而提高查询效率，并且也可以减少join的加工逻辑；但是维度退化也会导致一些问题，比如增大数据的存储量，维表进行更新的时候更新困难。

19. ods和dwd进行数据清洗的区别?

ods的数据清洗是轻度清洗，一般是对明显的异常值进行处理，比如主键和外键为空的情况，或者对字符串进行标准化处理。

dwd的数据清洗是会涉及数据验证，维度退化，以及将多张表进行关联操作等。

20. 如何评价数仓的好坏?

技术角度（又快又好，还要便宜安全）：

- 质量方面：是否存在过多的失败任务数；数据的准确性；数据的完整性；数据的一致性。
- 效率方面：SLA成功率；平均查询效应时间。
- 成本方面：处理的数据量和消耗的成本比值。
- 安全方面：高密表的热度，如果高密表被频繁访问说明安全性不高。

业务角度（全能易用，客户满意）：

- 覆盖方面：是否包含了各个业务流程所需要的数据。比如如果业务方需要获取区域销售信息来制定获得的时候我们能够及时提供数据，这就说明我们的覆盖比较全面。

- 易用方面：如果我们的公共层的数据能够支撑下游多个业务系统进行使用的话，说明我们的易用性较强。
- 客户满意程度：从使用客户的满意程度来体现我们数仓建设的好坏。
- 如何评价模型表的好坏？

模型设计原则契合度：

- 高内聚低耦合：业务类型相近的字段是否放在同一张表中，比如订单基础信息和订单支付信息。
- 核心模型与边缘模型分离：核心模型是否值放入了核心业务字段，比如订单ID，下单时间，下单金额。
- 公共逻辑下沉：统一的标准化处理等。
- 数据是否可回滚：多次运行的结果是否相同。

具体的指标：

- 模型的完整度：DWS层和ADS层对ODS层的引用率不能超过15%。
- 复用度：公共层的表是否被下层多次引用（>3次）。
- 成本和性能平衡：冗余字段不能超过核心字段的30%，DWS的任务运行时间小于1.5h。
- 如何设计一张宽表。
- 冷热分离：将经常使用的数据冗余到宽表中。
- 稳定和不稳定分离：将稳定表的字段进行冗余，不稳定的表字段留到ADS层进行冗余。这样不会影响到宽表的开发。
- 主次分类：当一个宽表的数据越来越多会失去主题性，这个时候将不同主题的字段进行分离，更加聚焦主题性，也便于开发维护。
- 怎么避免烟囱式开发？

烟囱式开发分为表级别烟囱式开发和字段级别烟囱式开发。

字段级别的可以建立指标字典，用于记录每个字段的口径和加工sql。

表级别的可以建立统一的元数据平台来对表信息进行统计，强化对于元数据信息的管理。

在开发规范上，要求开发之前都要去指标字典或者元数据平台去查看是否有一样的开发或者类似的开发。

24. 任务报错排查。

具体项目问题：

信贷整体业务流程：

展业：营销获客、引导账号注册和信息采集

授信：授信定额（动态调整-调整/冻结额度）、利率定价

贷前风控-额度-借款-贷中风控-放款-还款-催收简单回答一下授信业务过程

1. 信贷业务主要分为三个核心环节：贷前、贷中以及贷后。授信就属于贷前阶段。

- 首先用户注册,实名认证，绑定银行卡，然后提交授信申请，签署征信授权。
- 接着会有两轮校验，前置校验和反欺诈校验。前置校验会校验地区，年龄、反洗钱等基本信息。反欺诈校验是调用我们这边的反欺诈系统，判断恶意欺诈风险。
- 然后是信用审批，根据人行征信和外部大数据，A卡评分模型，确认是否审批通过。
- 最后根据申请的产品生成响应的额度。
- 在实习过程中遇到的最大问题是什么?

业务方面:

刚开始入职的时候对公司的整体业务流程不是很了解，有点无从下手的感觉。

首先就是梳理公司业务线的一个整体业务流程，比如在金融信贷中一般都会分成贷前-贷中-贷后三个阶段。

- 贷前：展业和授信。
- 贷中：额度，借款。
- 贷后：放款，还款，催收，

先了解整个业务流程，再细化到具体的域，这个时候我会主要通过公司沉淀的总线矩阵来帮助我进行梳理业务脉络。比如我之前在公司负责的是授信域，所以我会通过总线来看我负责的授信域主要有哪些业务流程，比如说授信申请，授信审批，利率定价等。

然后我会通过数据探查平台来对这些业务流程的表进行一个探查，看里面主要的字段，结合指标平台来统一字段的口径。

最后就是通过一些小的需求来进行上手开发，来进一步的巩固加深对业务流程的理解。

技术方面:

之前核心业务中的渠道风险报表出现了SLA报警处理，里面主要有渠道，借款金额，利率，30日违期总金额，不良率等字段。
在处理的时候主要是看yarn日志来追踪任务，发现在ruduce端出现一个任务执行时长远大于其他的任务，并且reduce的完成度一直卡在99%，这个时候就可以判断是出现了数据倾斜。
在处理的时候通过对渠道进行一个数据探查，在进行数据探查的时候发现unknown这个渠道字段的数量远大于其他有含义的渠道
首先第一个想到的是和业务那边进行确定看可不可以对渠道这个字段进行进一步挖掘，来对渠道进一步的划分，在得知不能之后；我使用负载均衡的思想，想着让更多的机器参加运算，我使用sql对热点值进行加盐处理然后和非热点值进行unoin all，之后再恢复原数据。
进行完优化之后报表的执行时间从2小时优化到1个半小时以内，成功解决了SLA产出问题。
经历这个事情之后我总结出了一套任务优化理论：查看yarn日志和解析执行计划来定位问题->数据探查发现热点值->和业务方尝试解决方案→sql代码优化。

3. 在实习过程中你最大的收获是什么?

在实习过程中总结出了一套自己的'性能问题排查与解决'的方法论。

首先通过查看yarn任务日志和解析执行计划来定位问题。

然后来解决具体的问题，比如数据倾斜问题的话会首先和业务区进行沟通，看能不能对里面的一些字段进行细化。如果不能的话就会通过一些常见的sql优化和参数优化来进行调优。

4. 面试之后的反问?

- 技术栈
- 这个岗位对候选人的关注点是什么？是更关注技术能力，还是团队协作能力?

- 团队有没有一些文档的沉淀？比如文档规范，指标口径？
- 你们公司的集群规模和开发环境是什么？

开发环境：DataX+Hadoop+Hive+Spark+Airflow+Doris

服务器配置：64core+256内存+12*8T硬盘。

数据存储规模：2-3PB，3副本机制。

任务个数：1000多个脚本，每天会有几千个app任务调度执行。

表分布个数：ODS 2000+， DWD 1600+， DWS 600+， ads 1000+，DIM 250+。

一个模糊的概念：2千万行数据，160个字段的客户信息表，每个日分区的大小3.5GB。

6. 数据倾斜问题有遇到过吗？是怎么解决的？

遇到过，数据倾斜的本质其实就是在shuffle的过程中数据分布不均匀导致的。

数据倾斜产生原因有很多，比如group by，count（distinct），join。

group by的优化方法：开启map端聚合。

join：使用skew join自动判断热点值或者手动将热点值进行加盐处理之后union到一起。

count（distinct）：在count之前将distinct的字段一起进行group by操作。

7. 需求承接/需求评审/需求排期

我们公司进行需求承接一般都是在jira上发布需求，其中会包括需求背景，需求内容，产出物是什么等。

如果需求比较简单的话就会直接和产品经理进行需求确定，然后就开始开发；如果需求比较复杂的话我们会开一个专门的需求评审会来确定需求，其中包括业务方，产品经理，数仓负责人，数据分析师和开发人员，主要确定需求的可行性，指标的口径，需求排期等内容。

8. 数据质量管理

待看视频。

9. spark和hive的任务怎么定位数据倾斜？

10. 项目拆解：

- 数仓业务指标，核心业务指标
- 数仓开发管理规范
- 技术栈的实战应用
- 如何定位排查sql问题？

首先需要通过调度平台来查看是什么问题，如果是一些简单的问题就可以直接解决了，比如上游表发生变更或者字段不存在等。

如果在调度平台上看不出问题再通过任务ID去yarn日志上查看（hive中一个stage对应一个application，spark一段sql对应一个application）。

hive任务直接通过查看application中的报错信息，查看是哪个stage出现了错误，通过explain来查看sql执行计划，根据过滤字段，扫描表等信息定位sql的位置进行定位排查。再对其进行sql调整，比如sql优化和相关参数优化。

spark任务同上。